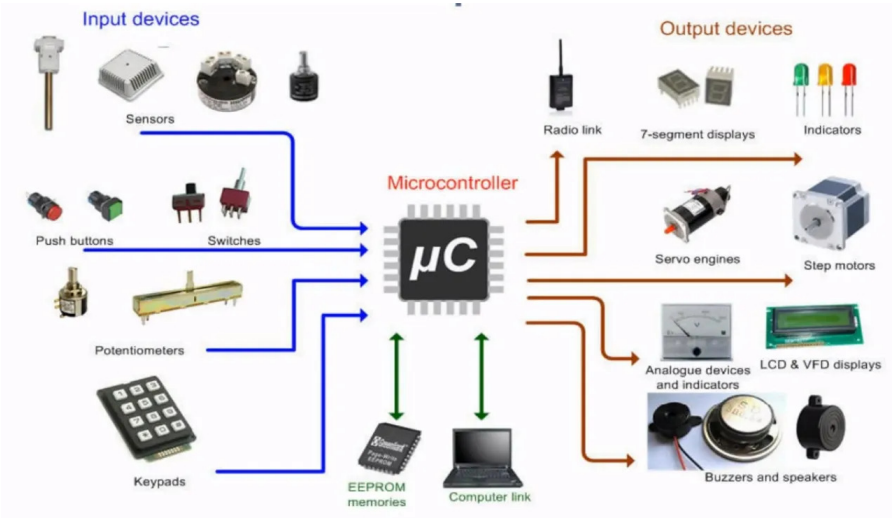
Architectures Logicielles et Matérielles

Cours 03 – Carte Discovery et environnement de développement

Bruno Ferres
basé sur un cours de Philippe Waille

10 septembre 2026

Rappel du contexte : les microcontrôleurs



Rôle des microcontrôleurs : une variété de signaux à gérer

Entrées

- capteurs analogiques : signal d'entrée vers un convertisseur analogique-numérique (*ADC*) ↪ microphone, pression, position, ...
- capteurs numériques renvoyant un entier via ports **parallèles** (plusieurs entrées booléennes)
  ↪ clavier, interrupteurs, capteur avec *ADC* intégré, ...

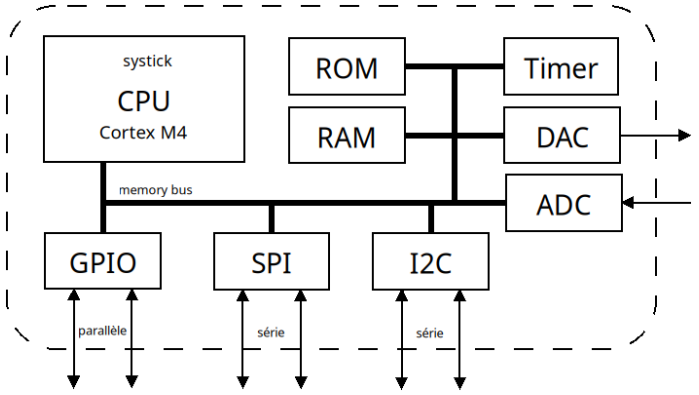
Sorties

- actionneurs analogiques : commande via un signal numérique converti (DAC) et amplifié ↪ haut-parleur, moteur continu, ...
- afficheurs et actionneurs numériques : commande via ports **parallèles** (plusieurs sorties booléennes)
  ↪ afficheur 7-segment, moteur pas à pas, électrovanne, ...

Mais aussi, capteurs/actionneurs numérique connectés par liaison **série**
↪ 1 ou 2 signaux de données (+ éventuellement horloge)
↪ RS232, SPI, I2C, Ethernet

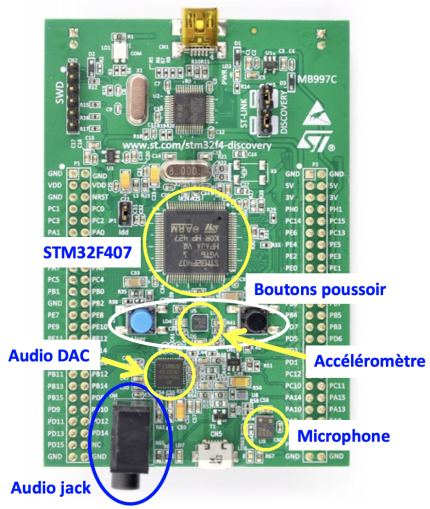
Synoptique du microcontrôleur de la carte Discovery

Microcontrôleur STM32F407 = CPU + ROM + RAM + E/S **monopuce**



Mémoire limitée : 1Mio de ROM et 128 kio + 64 kio de RAM

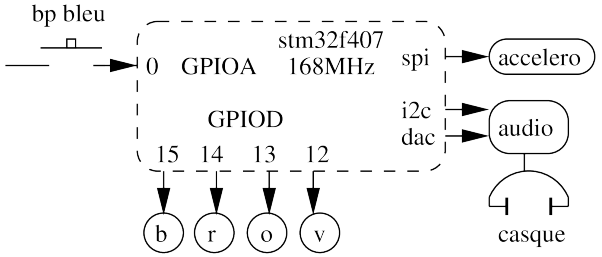
La carte Discovery F4



Préparation des TPs : périphériques externes

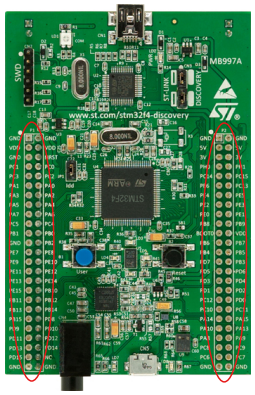
- bouton poussoirs sur `GPIOA0`
- LEDs (bleu, rouge, orange, vert) sur `GPIOD15` à 12
- accéléromètre via bus `SPI`
- circuit audio : configuration via bus `I2C`
  ↪ amplifie la sortie du DAC interne

mais c'est quoi, un **GPIO** ?



General Purpose Input/Output : les GPIO

Broches (physiques) d'interconnexion microcontrôleur ⇔ environnement :
↪ dont certaines pré-cablées, comme les LEDs, le bouton poussoir, ...



Broches (*pins*) de la carte Discovery F4

Couleur	GPIO	LED
Vert	PD12	LD4
Orange	PD13	LD3
Rouge	PD14	LD5
Bleu	PD15	LD6

Plus de signaux fonctionnels que de broches

Une histoire de coût

Le coût du microcontrôleur dépend entre autres du nombre de broches
↪ ∃ des versions de STM32F4 de 48 à 176 broches

Un grand nombre de périphériques internes...

- 3 ADC & 2 DAC
- 17 timers (générateurs signaux périodiques/horloge)
- connexion série : 3 `I2C`, 4 `RS232`, 3 `SPI`, 2 `USB`, 1 `Ethernet`
- interfaces SD card et LCD

... dont seuls quelques uns sont utilisés simultanément...
... et certains peuvent nécessiter plusieurs signaux !
↪ **ne pas attribuer de broche(s) dédiée(s) à chaque périphérique**

GPIO : le retour

Notion de GPIO

General **P**urpose **I**nput/**O**utput : entrées/sorties à usage général

Hors fonction fixes (alimentation, quartz, etc), les broches du STM32F407 sont regroupées en ports parallèles de 16 broches
↪ les GPIO (`GPIOA`, `GPIOB`, `GPIOC`, `GPIOD`, ...)

Configuration individuelle des broches

- 00 signal ordinaire (booléen) d'entrée : code `MODER 00`
- 01 signal ordinaire (booléen) de sortie : code `MODER 01`
- 11 signal analogique : code `MODER 11`
- 10 fonction spéciale (signal lié au fonctionnement d'un périphérique interne) : code `MODER 10`

Configuration avancée

Autres options de configuration non détaillées ici
↪ notions d'électronique nécessaires

- résistance de rappel à 0 (`PULLDOWN`) ;
- résistance de rappel à 1 (`PULLUP`) ;
- aucun rappel (`GPIO PUPD NONE` : cas usuel) ;
- sortie "*push-pull*" ou "*open drain*"
- option de vitesse

Adressage des périphériques internes

Périphérique dit *memory-mapped*

Un périphérique = un intervalle d'adresses (de taille 0x400)
cf *Memory Mapping* – documentation `rm0090` du STM32, page 53

À chaque registre E/S est associé un *offset* dans cet intervalle
↪ l'adresse du registre est la somme (*début intervalle* + *offset*)

Exemple du **R**andom **N**umber **G**enerator (RNG) page 591

[0x5006 0800, 0x5006 0*BFF*] (ou [0x5006 0800, 0x5006 0*C*00[)

registre	offset	adresse	fonction
`RNG_CR`	0x00	0x5006 0800	contrôle (`control register`)
`RNG_SR`	0x04	0x5006 0804	état (`status register`)
`RNG_DR`	0x08	0x5006 0808	donnée (`data register`)

Manipulation des périphériques GPIO

Trois principaux registres par GPIO page 198

1. `MODER` : *mode register* = configuration **individuelle** des broches
2. `IDR` : *input data register* = état des broches (si entrée)
3. `ODR` : *output data register* = état des broches (si sortie)

		`GPIOA`	`GPIOB`	`GPIOC`	`GPIOD`
	début	0x4002 0000	0x4002 0400	0x4002 0800	0x4002 0C00
registres	offset				
`MODER`	0x00	0x4002 0000	0x4002 0400	0x4002 0800	0x4002 0C00
`IDR`	0x10	0x4002 0010	0x4002 0410	0x4002 0810	0x4002 0C10
`ODR`	0x14	0x4002 0014	**0x4002 0414**	0x4002 0814	0x4002 0C14

↪ dans l'exemple, nous utiliserons **GPIOB_ODR**

Utilisation des GPIOs

Une utilisation en **3 temps**

1. activation (alimentation) + reset du GPIO
   ↪ en TP : fait dans le code qui vous est fourni
2. configuration **individuelle** du rôle de chaque broche
   ↪ **écriture** dans le registre `MODER`
3. si entrée : lecture du registre `IDR` (extraction de bits)
   si sortie : écriture du registre `ODR` (affectation de bits)

D'un point de vue pratique...

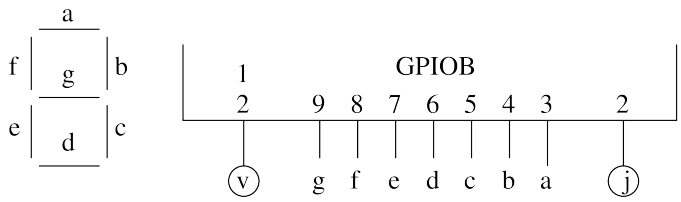
3 façons de manipuler possible, par difficulté décroissante :

1. directement : peu lisible
2. avec les macros de la bibliothèque `libopencm3`
3. avec les fonctions de la bibliothèque `libopencm3`

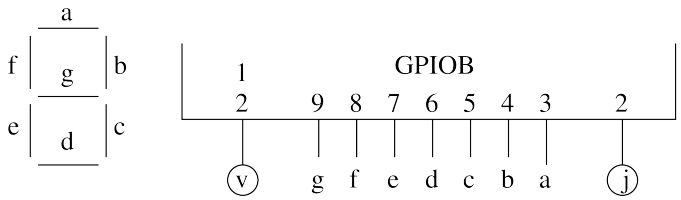
Exemple d'un afficheur 7 segment (carte fictive)

Connexions sur **GPIOB**

- une LED verte : broche 12
- une LED jaune : broche 2
- les LEDs a à g d'un afficheur 7 segments : broches 3 à 9



Affichage sur 7 segments



Comment afficher "1" **à gauche** (inhabituel) ?

Segments à allumer : e et f
↪ forcer `GPIOB7` et `GPIOB8` à 1, **les autres segments à 0**
↪ valeur à écrire (en hexa) : `0x0180`

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	numéros bits
0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	valeurs

Afficheur 7 segments – sans `libopencm3` (1/2)

Première solution

Affectation au registre ODR du GPIOB

```
*(volatile uint32_t *)0x40020414 = 0x0180;
// 32 bits, mais 16 bits de poids forts doivent rester à 0
```

Problèmes :

- peu lisible
- si l'adresse et/ou les numéros de ports changent...
  ↪ Ctrl-F et on replace tout !
  ↪ problème de **portabilité**
- affectes les LEDs jaune (broche 2) et verte (broche 12)

Afficheur 7 segments – sans libopencm3 (2/2)

Deuxième solution

```
#define BITA 3
#define BITB 4
#define BITC 5
#define BITD 6
#define BITE 7
#define BITF 8
#define BITG 9

*(volatile uint32_t *)0x40020414 = (1<<BITE)|(1<<BITF);
```

↪ un peu plus lisible, mais affecte toujours les LEDs !

Afficheur 7 segments – avec macros (1/2)

Macros définies par la bibliothèque libopencm3

La libopencm3 contient l'équivalent de :

- `#define GPIOB_ODR (*(volatile uint32_t *)0x40020414)`
  - ↪ GPIOB_ODR s'utilise comme une variable C
  - ↪ **attention** : les 16 bits de poids forts doivent rester à 0 !
-
```
#define GPIO0 (1<<0)
#define GPIO1 (1<<1)
#define GPIO2 (1<<2)
... // etc
#define GPIO15 (1<<15)
```

```
GPIOB_ODR = (GPIO7)|GPIO8);
```

↪ encore plus lisible et portable (mais peu mieux faire) !

Afficheur 7 segments – avec macros (2/2)

Encore plus de macros

```
#define SEGA GPIO3
#define SEGB GPIO4
#define SEGC GPIO5
#define SEGD GPIO6
#define SEGE GPIO7
#define SEGF GPIO8
#define SEGG GPIO9

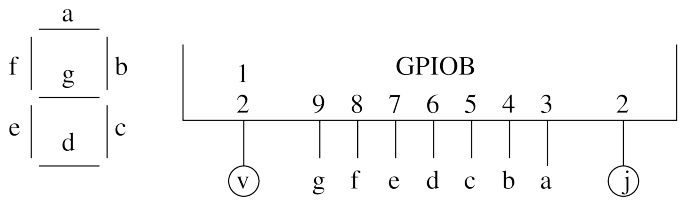
#define ALL_SEGS (SEGA|SEGB|SEGC|SEGD|SEGE|SEGF|SEGG)
```

Enfin un code lisible (et portable) :

```
GPIOB_ODR = SEGE | SEGF;
```

↪ **mais affecte toujours les LEDs jaune et verte !**

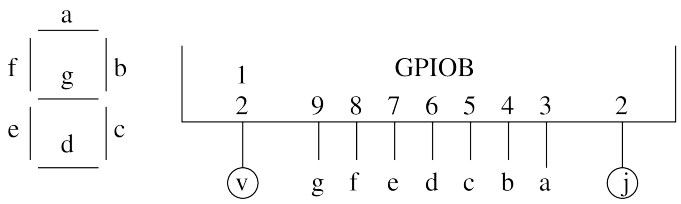
Affecter les segments sans modifier les LEDs



Évolution de l'état des broches

- état initial : a, e et LED verte allumés, le reste éteint
- état intermédiaire : LED verte allumée, le reste éteint
  - ↪ obtenu grâce à un ET bit à bit (&) avec le bon masque
- état final : e, f et LED verte allumés, le reste éteint
  - ↪ obtenu grâce à un OU bit à bit (|) avec le bon masque

Détail des masques



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	*1*	0	0	0	0	1	0	0	0	1	*0*	0	0	état initial
1	1	1	1	1	1	**0**	**0**	**0**	**0**	**0**	**0**	**0**	1	1	1	forçage à 0
0	0	0	*1*	0	0	0	0	0	0	0	0	0	*0*	0	0	intermédaire
0	0	0	0	0	0	0	**1**	**1**	0	0	0	0	0	0	0	forçage à 1
0	0	0	*1*	0	0	0	1	1	0	0	0	0	*0*	0	0	état final

0x1088 $\xrightarrow{\&_{bb}\ 0xfc07}$ 0x1000 $\xrightarrow{|_{bb}\ 0x0180}$ 0x1180

Une fonction d'affectation utilisant les masques

```
#define ALL_SEGS (SEGA|SEGB|SEGC|SEGD|SEGE|SEGF|SEGG)

void write_segments(uint16_t segmask) {
    uint16_t value;
    value = GPIOB_ODR;
    value = value & ~ALL_SEGS;
    value = value | segmask;
    GPIOB_ODR = value;
}

// Utilisation
write_segments(SEGE|SEGF);
```


Variante : fonction d'affectation par entier (1/2)

Variante de la spécification

le paramètre est un entier $\in [0, 127]$:

- le bit 0 définit l'état du segment a ;
- le bit 1 définit l'état du segment b ;
- ...
- le bit 6 définit l'état du segment g ;

```
void write_segments_int(uint16_t mask) {
    uint16_t value = GPIOB_ODR;
    value = value & ~ALL_SEGS;
    GPIOB_ODR = value | (mask << BITA);
}
```


Variante : fonction d'affectation par entier (2/2)

Utilisation

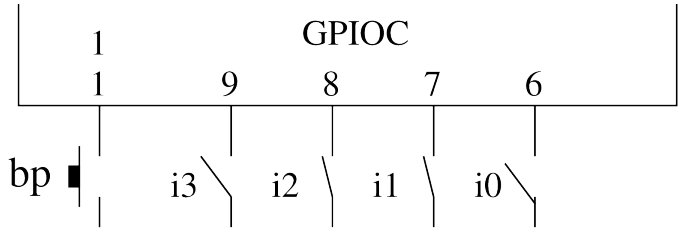
```
#define A (1<<0)
#define B (1<<1)
#define C (1<<2)
#define D (1<<3)
#define E (1<<4)
#define F (1<<5)
#define G (1<<6)

// utilisation
write_segments_int(F|G);
```

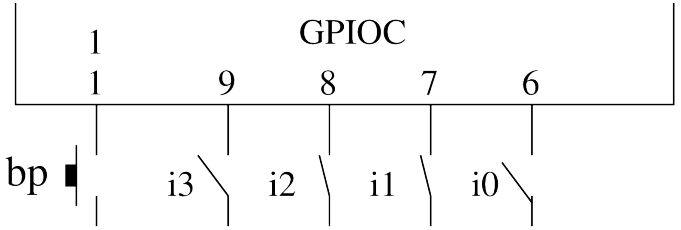
Exemple de manipulation d'interrupteurs (carte fictive)

Connexions sur **GPIOC**

- 4 interrupteurs : broches 6 à 9
- un bouton poussoir : broche 11



Accès aux valeurs

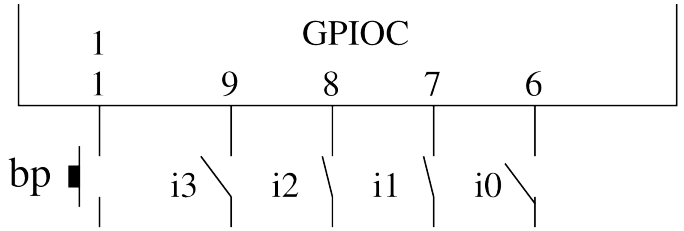


Comment accèder aux valeurs saisies sur les entrées

- comment récupérer l'entier ($\in [0, 15]$) codé sur les 4 interrupteurs ?
  ↪ valeur 6 (= 0b0110) sur la figure
- comment récupérer l'état ($\in [0, 1]$) du bouton poussoir ?
  ↪ 0 (= 0b0) sur la figure

↪ **solution : lire `GPIOC_IDR` + décalage et masquage**

Décalage et masquage pour les interrupteurs – déroulé



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
?	?	?	?	0	?	0	1	1	0	?	?	?	?	?	?	lecture IDR
?	?	?	?	?	?	?	?	?	?	0	?	0	1	1	0	IDR $\gg$ 6
0	0	0	0	0	0	0	0	0	0	0	0	**1**	**1**	**1**	**1**	& 0xf
0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	valeur finale

Décalage et masquage pour les interrupteurs – code

```
#define SWITCHS_POSITION 6
#define SWITCHS_MASK (0xf << SWITCHS_POSITION)

uint16_t read_inters_v1 (){
    uint16_t value = GPIOC_IDR;
    value = value >> SWITCHS_POSITION;
    return value & 0xf;
}

uint16_t read_inters_v2 (){
    uint16_t value = GPIOC_IDR;
    value = value & SWITCHS_MASK;
    return value >> SWITCHS_POSITION;
}
```

pourquoi 0xF ? pourquoi 6 ?

Décalage et masquage pour le bouton poussoir

```
#define PUSHBUTTON_POSITION 11
#define PUSHBUTTON_MASK (1 << PUSHBUTTON_POSITION)

uint16_t read_bp_v1 (){
    uint16_t value = GPIOC_IDR;
    value = value >> PUSHBUTTON_POSITION;
    return value & 1;
}

uint16_t read_bp_v2 (){
    uint16_t value = GPIOC_IDR;
    value = value & PUSHBUTTON_MASK;
    return value >> PUSHBUTTON_POSITION;
}
```


Quelques fonctions GPIO (E/S) de la libopencm3

- lecture de la donnée d'un port :
  `valeur = gpio_port_read(GPIOC);`
  équivalent : `valeur = GPIOC_IDR;`
- écriture de la donnée d'un port :
  `gpio_port_write(GPIOC, valeur);`
  équivalent : `GPIOC_ODR = valeur;`
- forçage de broches à 1 :
  `gpio_set(GPIOD, GPIO10|GPIO1);`
  équivalent : `GPIOC_ODR = GPIO_ODR | (GPIO10|GPIO1);`
- forçage de broches à 0 :
  `gpio_clear(GPIOD, GPIO8|GPIO1);`
  équivalent : `GPIOC_ODR = GPIO_ODR & ~(GPIO8|GPIO1);`

Configuration individuelle des broches : le registre MODER

Rôle du registre MODER

permet de définir **individuellement** le rôle de chaque broche d'un port
↪ configuration : 16 paires de bits du registre MODER à définir

31 30	29 28	27 26	25 24	23 22	21 20	19 18	17 16
MODER15[1 :0]	MODER14[1 :0]	MODER13[1 :0]	MODER12[1 :0]	MODER11[1 :0]	MODER10[1 :0]	MODER 9[1 :0]	MODER 8[1 :0]
rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw

15 14	13 12	11 10	9 8	7 6	5 4	3 2	1 0
MODER 7[1 :0]	MODER 6[1 :0]	MODER 5[1 :0]	MODER 4[1 :0]	MODER 3[1 :0]	MODER 2[1 :0]	MODER 1[1 :0]	MODER 0[1 :0]
rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw	rw rw

Exemple

Broches 0 et 1 analogiques (11), broches 2 et 14 en sortie (01), broches 3 fonction de périphérique (10), autres broches en entrée (00)

$$\underbrace{00}_{e_{15}}\ \underbrace{01}_{s_{14}}\ \underbrace{0000}_{e_{13}\ e_{12}}\ \underbrace{0000\ 0000\ 0000}_{e_{11}\ \dots\ e_6}\ \underbrace{00}_{e_5}\ \underbrace{00}_{e_4}\ \underbrace{10}_{p_3}\ \underbrace{01}_{s_2}\ \underbrace{11}_{a_1}\ \underbrace{11}_{a_0}$$

`GPIOC_MODER = 0x1000009f`

Configurer les broches avec la libopencm3

```
// rappel : broches en entrée au reset (mode par défaut)

gpio_mode_setup(GPIOC,GPIO_MODE_OUTPUT,
                GPIO_PUPD_NONE,GPIO14|GPIO2);

gpio_mode_setup(GPIOC,GPIO_MODE_ANALOG,
                GPIO_PUPD_NONE,GPIO1|GPIO0);

gpio_mode_setup(GPIOC,GPIO_MODE_AF,GPIO_PUPD_NONE,GPIO3);

// choisir le périphérique de la fonction spéciale
gpio_set_af(GPIOC, ..., GPIO3)
```

Modification de bits "atomiques"

Principe d'atomicité

Possibilité de modifier l'état d'une ou plusieurs broche(s) avec
un seul accès (*atomique*) en écriture : registre BSRR (*bit set and reset register*)

Lors d'une écriture dans `BSRR` :

- chacun des bits b de poids faibles ($b \in [0, 15]$) à 1 de la valeur écrite **force** la broche b correspondante à 1 (sans affecter les autres broches)
- chacun des bits b de poids forts ($b \in [16, 31]$) à 1 de la valeur écrite **force** la broche $b - 16$ correspondante à 0
- si bits b et $b + 16$ sont tous les 2 à 1, la broche b est **forcée** à 1

Exemples

- forcer les broches 5 et 3 à 1 : BSRR $\leftarrow (1 \ll 5 \mid 1 \ll 3)$
- forcer les broches 5 et 3 à 0 : BSRR $\leftarrow (1 \ll (5+16) \mid 1 \ll (3+16))$
- forcer les broches 5 à 1 et 3 à 0 : BSRR $\leftarrow (1 \ll 5 \mid 1 \ll (3+16))$

Préparation des TPs : la routine `write_leds`

Rappel sur la carte Discovery

- bleue : GPIOD15
- rouge : GPIOD14
- orange : GPIOD13
- verte : GPIOD12

Spécification

- ne **doit pas** modifier l'état des autres broches du `GPIOD`
- le paramètre est un entier ($\in [0, 15]$) décrivant quelles LEDs allumer

↪ ex. : $0x9 = 1001_2$ allume bleue et verte et éteint les autres

Implémentation en utilisant

1. uniquement le registre `GPIOD_ODR`
2. uniquement le registre `GPIOD_BSRR`

TIMERs avec signal externe

Qu'est ce qu'un TIMER ?

compteur/décompteur cadencé à un sous-multiple de la fréquence du processeur (ici, 84MHz/prédiviseur)

Utilisation d'un TIMER sur signal d'entrée

permet de mesure un signal booléen d'un capteur, par exemple :

- durée d'une impulsion
- la période (1/fréquence) d'un signal périodique (ex : vitesse de rotation)
- le rapport cyclique (durée haut/durée période) d'un signal périodique

Utilisation d'un TIMER sur signal de sortie

permet de générer une commande booléenne d'actionneur, par exemple :

- une impulsion de durée donnée (ex : freiner pendant 1 milliseconde)
- un signal périodique de rapport cyclique et de fréquence donnés
  ↪ génération d'un signal d'horloge **pour le circuit audio**

TIMERS sans signal externe et systick

TIMER sans signal externe : génération **périodique** d'évènements

- à chaque évènement, forcer l'exécution d'une routine de traitement de l'évènement
  ↪ mécanisme d'interruption : cf prochain cours
- le `systick` est un timer interne au CPU dépourvu de signal externe

Exemples d'application

- générer un évènement à une fréquence de 1Hz et incrémenter l'heure courante de 1s à chaque évènement
- **cadencer** un jeu de lumières de type "chenillard"

Exemple de configuration :

```
timer_set_period(TIM3,period);
systick_set_reload(reload_val);
```

Générateur de nombres aléatoires (RNG)

Génération logicielle **déterministe**

La génération de nombres par la bibliothèque `random` est déterministe
↪ la suite de nombres est entièrement définie et prévisible :
$u_0 = seed$; $u_{n+1} = f(u_n)$
↪ inadapté à certaines applications, **notamment la cryptographie**

Génération "réellement" aléatoire

Le **R**andom **N**umber **G**enerator est un générateur d'une suite de nombres effectivement aléatoire basée sur le "bruit" électronique des circuits
↪ génère un nombre toutes les 48 périodes de son horloge
↪ peut tomber dans un état trop stable à entropie anormale, auquel cas il faut le redémarrer

Convertisseur Numérique/Analogique (DAC)

Rôle d'un DAC ou CNA

- un canal convertit un entier sur 8 ou 12 bits en tension analogique.
- utilisé pour générer (logiciel) un signal périodique en dents de scie
  ↪ à amplifier par le circuit audio externe.

Caractéristiques

- 2 canaux
- $V_{sortie} = V_{dd} \times \frac{x}{4096}$
- Temps de conversion typique : $3\mu s$

Exemple d'utilisation :

```
dac_load_data_buffer_single(DAC1,valeur,
                            DAC_ALIGN_RIGHT12,DAC_CHANNEL1);
```


À la découverte de l'API de la bibliothèque `libopencm3`

Ressources en ligne

https://libopencm3.org/docs/latest/stm32f4/html/modules.html

- STM32F4 → Peripheral APIs :
  fonction de GPIO, SPI, I2C, TIMERS, DAC
- STM32F4 → STM32F4xx Defines :
  les `#define` des constantes de périphérique
- STM32F4 → Cortex Core Peripheral APIs :
  les fonctions pour `systick` et le `NVIC` (interruptions)
- STM32F4 → CM3 Defines → Cortex Core Defines :
  fonction de gestion d'interruption au niveau du CPU

Exercice

On considère l'afficheur 7 segments présenté plus tôt.

Écrire les expressions `C` permettant d'afficher les caractères `A` et `B` :

1. en écrivant directement sur le port `GPIOB_ODR` ;
2. en utilisant la fonction définie en slide 27
   `void write_segments_int(uint16_t mask)`

Sémantique d'un signal : niveau (valeur) *vs* front

Cas d'usage illustratif — Horloge logicielle

Ecrire un programme qui met à jour l'heure courante (`HH:MM:SS`)

```
while(1) {
    attendre_une_seconde();
    secondes++;
    if (secondes >= 60) {
        secondes = 0;
        minutes++;
        if (minutes >= 60) {
            minutes = 0;
            heures++;
            if (heures >= 24) {
                heures = 0;
            }
        }
    }
}
```

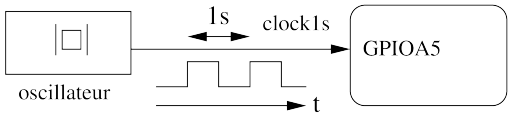
Implémentation

Comment coder `attendre_une_seconde` ?

Récupération de la valeur d'une horloge

On connecte un oscillateur à 1 hertz qui génère un signal périodique carré `clock1s` d'une période de une seconde à la broche `GPIOA5`

Signal carré : états haut et bas de durée égale (de 500ms chacun).



Questions

- écrire une fonction `get_clock()` qui retourne l'état (0 ou 1) du signal `clock1s`
- écrire la fonction `attendre_une_seconde`

Fonction `get_clock`

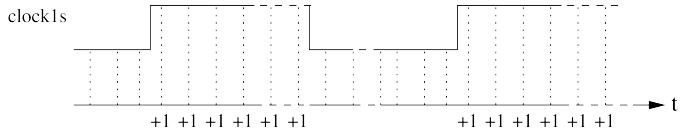
```
#define CLOCK1S_BIT 5
#define CLOCK1S_MASK (1 << CLOCK1S_BIT)

// Version 1 avec numéro de bit
int get_clock () {
    return (GPIOA_IDR >> CLOCK1S_BIT) & 1;
}

// Version2 avec masque
int get_clock () {
    return (GPIOA_IDR & CLOCK1S_MASK) != 0;
}
```

↪ **renvoie la valeur du signal d'horloge**

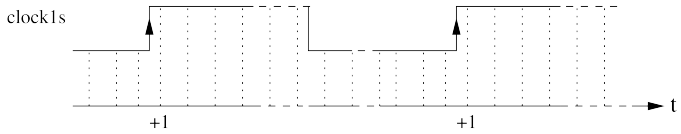
Le **niveau** (valeur) de `clock1s` n'est pas pertinent



Échelle de temps dilatée

```
void attendre_une_seconde() {
    return get_clock();                    // CODE FAUX !!
}
```

1 seconde = **front** (montant) de `clock1s`



Échelle de temps dilatée

```
void attendre_une_seconde() {
    // attendre le prochain front montant
    while (get_clock() != 0) {}      // attente d'un niveau bas
    while (get_clock() == 0) {}      // attente d'un niveau haut
}
```


De la transmission de données au contrôle de flux

Que sont les entrées/sorties ?

échange entre CPU+mémoire ⇔ environnement
↪ débit pas forcément régulier, et dépendant de l'environnement
(ex : clavier)

Comment gérer cette dépendance à l'extérieure ?

- ajouter deux signaux de **contrôle de flux** entre émetteur et récepteur :

récepteur ←*entrée*— émetteur
Processeur + Mem environnement (**périphérique**)
émetteur —*sortie*→ récepteur

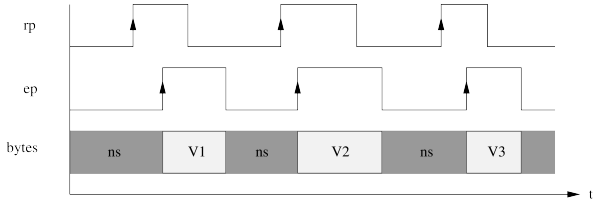
- **protocole de synchronisation** de type "poignée de mains" (*handshake*)

Signaux de synchronisation pour une poignée de main

Pourquoi a-t-on besoin de deux signaux ?

- un signal donnée/émetteur prêt nécessaire :
  - de l'émetteur vers le récepteur
  - pour distinguer des séquences uniformes :
    {1,1,1} *vs* {1} *vs* {1,1,1,1,1}
  - pour éviter que le récepteur duplique d"s données
- un signal récepteur prêt nécessaire :
  - du récepteur vers l'émetteur
  - pour empêcher l'émetteur d'envoyer des données plus vite que le récepteur ne peut gérer
  - évite donc la perte de données
- chaque signal doit effectuer un cycle complet pour que l'émetteur et le récepteur passent à la donnée suivante

Chronogramme d'un échange



rp : récepteur prêt, ep : émetteur prêt, ns : non significatif

Protocole pour échanger une donnée

1. *rp* == 1 et *ep* == 0 : émission de la donnée courante possible
   ↪ réponse au front montant de *rp*
2. *rp* == 1 et *ep* == 1 : donnée courante émise, réception possible
   ↪ réponse au front montant de *ep*
3. *rp* == 0 et *ep* == 1 : donnée courante émise et reçue
4. *rp* == 0 et *ep* == 0 : transfert entièrement terminé (état initial)

Code d'un récepteur

État initial

Transfert précédent terminé : $ep == rp == 0$

Réception d'une donnée (Pseudo-algorithme)

```
1 rp ← 1;
2 tantQue (get_ep() == 0)  // attente front montant ep
3 lu ← data;
4 rp ← 0;
5 tantQue (get_ep() != 0)  // attente front descendant ep
6 renvoyer lu;
```

Implémentation

On peut ajouter un peu de circuiterie pour réaliser automatiquement (c.a.d. **en matériel**) les étapes 1, 4 et 5, déclenchées par la lecture à l'étape 3 et le front descendant de ep.

Code d'un émetteur

État initial

Transfert précédent terminé : $ep == rp == 0$

Émission d'une donnée d (Pseudo-algorithme)

```
1 tantQue (get_rp() == 0)  // attente front montant rp
2 data ← d;
3 ep ← 1;
4 tantQue (get_rp() != 0)  // attente front descendant rp
5 ep ← 0;
```

Implémentation

On peut ajouter un peu de circuiterie pour réaliser automatiquement (c.a.d. **en matériel**) les étapes 3 à 5, déclenchées par l'écriture à l'étape 3 et le front descendant de rp.